CSA4704 – DEEP LEARNING

Name: PAVITHRA K Register Number:192411027

EXPERIMENT 1: VECTOR AND MATRIX OPERATIONS

Aim: To perform vector and matrix operations using NumPy and understand how matrix operations are used in a simple recommendation system.

Algorithm

Algorithm:

1. Import NumPy and Matplotlib.
2. Create a customer-genre matrix and a genre-book matrix.
3. Perform dot product, matrix multiplication, transpose and element-wise operations.
4. Calculate cosine similarity between customers.
5. Apply a simple dense-layer operation and display a heatmap.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Rows represent customers; columns represent genres
customer_genre = np.array([
    [5, 1, 0, 2, 1],
    [4, 2, 1, 0, 2],
    [0, 1, 5, 4, 1],
    [1, 0, 4, 5, 2],
    [2, 4, 1, 0, 5],
    [1, 3, 2, 1, 4]
], dtype=float)

# Rows represent genres; columns represent books
genre_book = np.array([
    [5, 2, 1, 0],
    [1, 4, 2, 1],
    [0, 1, 5, 3],
    [1, 0, 4, 5],
    [2, 3, 1, 4]
], dtype=float)

print("Dot product:", np.dot(customer_genre[0], genre_book[:, 0]))
print("Matrix multiplication:\n", customer_genre @ genre_book)
print("Transpose:\n", customer_genre.T)
print("Element-wise square:\n", customer_genre ** 2)

# Cosine similarity between first two customers
a, b = customer_genre[0], customer_genre[1]
similarity = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print("Cosine similarity:", round(similarity, 3))

# Simple dense layer: y = Wx + bias
W = np.array([[0.2, 0.5], [0.4, 0.3]])
x = np.array([1.0, 2.0])
bias = np.array([0.1, 0.2])
print("Dense layer output:", W @ x + bias)
```

```
plt.imshow(customer_genre, cmap="Blues", aspect="auto")
plt.title("Customer-Genre Matrix")
plt.xlabel("Genres")
plt.ylabel("Customers")
plt.colorbar()
plt.show()
```



Result

Result: Vector and matrix operations, cosine similarity and a simple dense-layer output were calculated successfully.

EXPERIMENT 2: COVARIANCE, EIGENVALUES AND EIGENVECTORS

Aim: To analyse a dataset using covariance, correlation, eigenvalues and eigenvectors, and reduce its dimensions using PCA.

Algorithm

Algorithm:

1. Generate or load numerical data.
2. Standardise the features.
3. Calculate covariance and correlation matrices.
4. Find eigenvalues and eigenvectors of the covariance matrix.
5. Sort components by eigenvalue and project the data into two dimensions.
6. Plot the reduced data.

Python Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
np.random.seed(10)
data = np.random.randn(200, 3)
data[:, 1] = 0.7 * data[:, 0] + 0.5 * data[:, 1]
data[:, 2] = 0.4 * data[:, 0] + 0.6 * data[:, 2]

# Standardisation
X = (data - data.mean(axis=0)) / data.std(axis=0)

cov_matrix = np.cov(X, rowvar=False)
corr_matrix = np.corrcoef(X, rowvar=False)

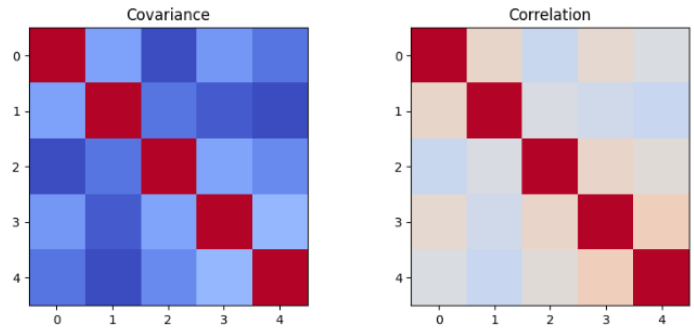
print("Covariance matrix:\n", np.round(cov_matrix, 3))
print("Correlation matrix:\n", np.round(corr_matrix, 3))

# Eigen decomposition
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

variance_percent = eigenvalues / eigenvalues.sum() * 100
print("Eigenvalues:", np.round(eigenvalues, 3))
print("Variance explained (%):", np.round(variance_percent, 2))

# Project data onto first two principal components
X_2d = X @ eigenvectors[:, :2]
plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.6)
plt.title("PCA: Data in Two Dimensions")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid()
plt.show()
```

..




```
Eigenvalues: [1.32511405 1.11031401 0.9828869  0.83771742 0.74396762]
Variance: [26.50228096 22.20628013 19.65773808 16.7543484  14.87935244]
```



Result

Result: Covariance and correlation matrices were obtained. Eigenvalues and eigenvectors were used to identify the principal components and reduce the data to two dimensions.

EXPERIMENT 3: PROBABILITY DISTRIBUTIONS AND BAYES' THEOREM

Aim

To generate standard probability distributions and calculate the probability that a patient has a disease after receiving a positive test result using Bayes' theorem.

Algorithm

1. Import NumPy, SciPy and Matplotlib.
2. Generate Bernoulli, Binomial, Poisson, Uniform and Gaussian samples and compare them with their probability functions.
3. Calculate the posterior probability using Bayes' theorem.
4. Simulate 100,000 patients and compare the empirical result.
5. Plot posterior probability against disease prevalence.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import bernoulli, binom, poisson, uniform, norm



np.random.seed(42)

dists = [(bernoulli, (0.5,), "Bernoulli"),

        (binom, (10, 0.5), "Binomial"),

        (poisson, (4,), "Poisson"),

        (uniform, (0, 1), "Uniform"),

        (norm, (0, 1), "Gaussian")]



fig, ax = plt.subplots(1, 5, figsize=(15, 3))

for a, (d, p, name) in zip(ax, dists):

   sample = d.rvs(*p, size=1000)

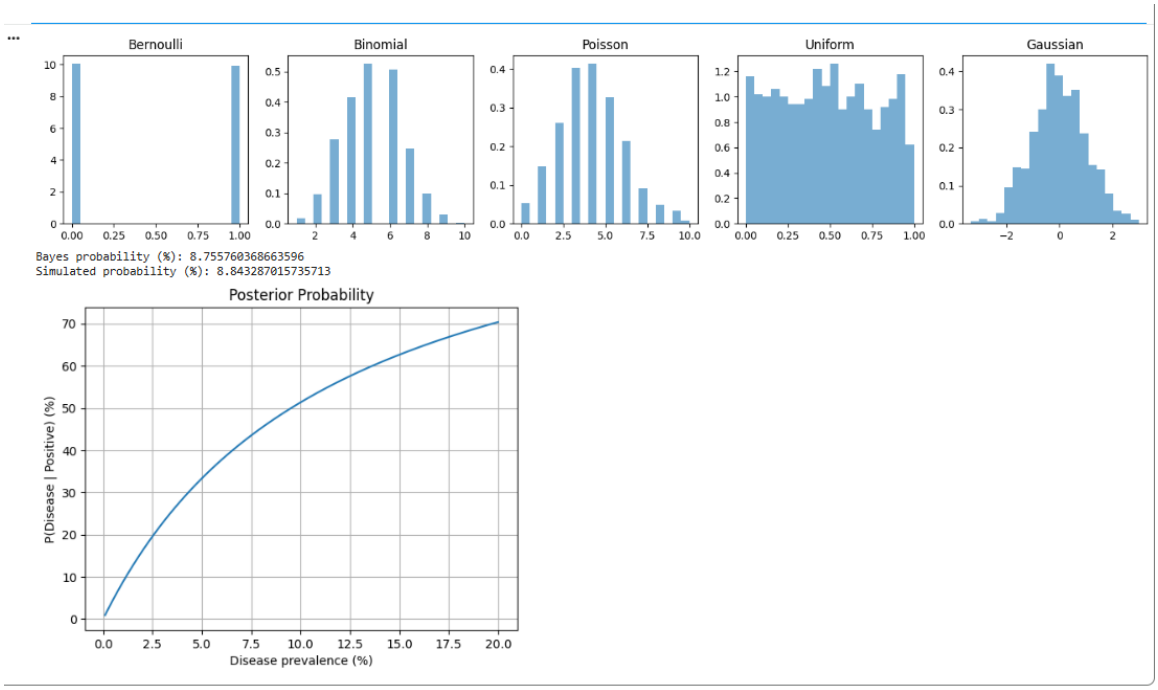
   a.hist(sample, bins=20, density=True, alpha=0.6)
```

```
    a.set_title(name)
plt.tight_layout()
plt.show()


# Bayes' theorem
prev, sensitivity, specificity = 0.01, 0.95, 0.90
posterior = (prev*sensitivity) / (
    prev*sensitivity + (1-prev)*(1-specificity))
print("Bayes probability (%):", posterior*100)


# Simulation of 100,000 patients
disease = np.random.rand(100000) < prev
positive = np.where(disease,
    np.random.rand(100000) < sensitivity,
    np.random.rand(100000) < (1-specificity))
print("Simulated probability (%):",
      disease[positive].mean()*100)


# Posterior probability for different prevalence values
p = np.linspace(0.001, 0.20, 100)
post = p*sensitivity / (p*sensitivity + (1-p)*(1-specificity))
plt.plot(p*100, post*100)
plt.xlabel("Disease prevalence (%)")
plt.ylabel("P(Disease | Positive) (%)")
plt.title("Posterior Probability")
plt.grid()
plt.show()
```



Result

Thus, standard probability distributions were generated, Bayes' theorem was applied, and the theoretical posterior probability was compared with a simulation of 100,000 patients.

EXPERIMENT 4: ENTROPY, CROSS-ENTROPY AND KL DIVERGENCE

Aim

To calculate entropy, cross-entropy and KL divergence for weather prediction models and relate them to classification loss.

Algorithm

6. Import NumPy, SciPy and Matplotlib.
7. Define the true weather distribution and two model predictions.
8. Calculate entropy, cross-entropy and KL divergence.
9. Verify that cross-entropy equals entropy plus KL divergence.
10. Calculate KL divergence in both directions to show asymmetry.
11. Plot binary cross-entropy loss against predicted probability.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import entropy
```

```
P = np.array([0.5, 0.3, 0.2])       # True: Sunny, Cloudy, Rainy
M1 = np.array([0.6, 0.25, 0.15])
M2 = np.array([0.3, 0.4, 0.3])

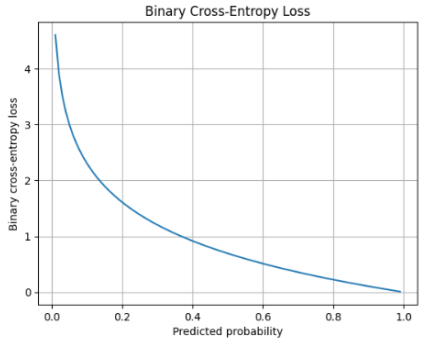

H = entropy(P)
print("Entropy:", H)


for M in [M1, M2]:
    CE = -np.sum(P*np.log(M))
    KL = entropy(P, M)
    print("Cross-entropy:", CE)
    print("KL divergence:", KL)
    print("Entropy + KL:", H + KL)


print("KL(P || M1):", entropy(P, M1))
print("KL(M1 || P):", entropy(M1, P))


# Binary cross-entropy when the true label is 1
prob = np.linspace(0.01, 0.99, 100)
loss = -np.log(prob)
plt.plot(prob, loss)
plt.xlabel("Predicted probability")
plt.ylabel("Binary cross-entropy loss")
plt.title("Binary Cross-Entropy Loss")
plt.grid()
plt.show()
```

```
Entropy: 1.0296530140645737
Cross-entropy: 1.0507251171961387
KL divergence: 0.02107210313156528
Entropy + KL: 1.050725117196139
Cross-entropy: 1.1176681825904018
KL divergence: 0.0880151685258282
Entropy + KL: 1.1176681825904018
KL(P || M1): 0.02107210313156528
KL(M1 || P): 0.02066023401011695
```



Result

Thus, entropy, cross-entropy and KL divergence were calculated for two weather prediction models. The relationship between the measures and the asymmetry of KL divergence were demonstrated.

EXPERIMENT 5: BIAS–VARIANCE, OVERFITTING AND UNDERFITTING ANALYSIS

Aim

To compare polynomial regression models of different degrees and identify underfitting, good fitting and overfitting using Scikit-learn.

Algorithm

12. Generate 150 noisy house-area and house-price samples.
13. Split the dataset into training and testing sets.
14. Fit polynomial regression models of degrees 1, 3, 9 and 15.
15. Calculate training and testing mean squared errors.
16. Plot the fitted curves and error against model complexity.
17. Select the degree with the lowest test error.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.pipeline import make_pipeline

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error
```

```
np.random.seed(42)
X = np.linspace(500, 2500, 150).reshape(-1, 1)
y = 1000 + 20*X.ravel() + 0.01*X.ravel()**2
y += np.random.normal(0, 15000, 150)

Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=42)

degrees = [1, 3, 9, 15]
train_errors, test_errors = [], []

for d in degrees:
    model = make_pipeline(PolynomialFeatures(d), LinearRegression())
    model.fit(Xtr, ytr)
    train_errors.append(mean_squared_error(ytr, model.predict(Xtr)))
    test_errors.append(mean_squared_error(yte, model.predict(Xte)))
    plt.plot(X, model.predict(X), label="Degree "+str(d))

plt.scatter(X, y, s=10, color="black")
plt.xlabel("House area")
plt.ylabel("House price")
plt.title("Polynomial Regression")
plt.legend()
plt.show()

plt.plot(degrees, train_errors, "o-", label="Training error")
plt.plot(degrees, test_errors, "o-", label="Testing error")
```

```
plt.xlabel("Polynomial degree")

plt.ylabel("Mean squared error")

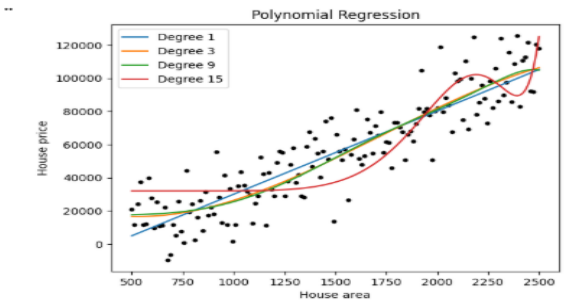
plt.legend()

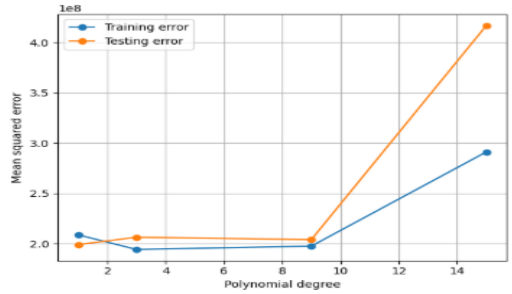
plt.grid()

plt.show()



print("Best degree:", degrees[np.argmin(test_errors)])
```






```
Best degree: 1
```

Result

Thus, polynomial models of different degrees were trained and compared. Training and testing errors were plotted to study underfitting and overfitting and select a suitable model.

EXPERIMENT 6: GRADIENT DESCENT, SGD, MOMENTUM AND ADAM COMPARISON

Aim

To implement and compare gradient descent, stochastic gradient descent, momentum and Adam optimisers using optimisation paths and training loss curves.

Algorithm

18. Define a simple two-variable loss function and its gradient.
19. Implement GD, SGD and Momentum from scratch.
20. Record the parameter updates and plot optimiser paths.
21. Create a small regression dataset.
22. Train the same Keras model using SGD, SGD with Momentum and Adam.

23. Plot loss against epochs and compare convergence.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf


def grad(w):

    return np.array([2*w[0], 4*w[1]])


# Optimisers from scratch

for name in ["GD", "SGD", "Momentum"]:

    w = np.array([4.0, 4.0])

    v = np.zeros(2)

    path = [w.copy()]


    for i in range(80):

        g = grad(w)

        if name == "SGD":

            g += np.random.normal(0, 0.2, 2)

        if name == "Momentum":

            v = 0.9*v + 0.1*g

            w -= 0.1*v

        else:

            w -= 0.1*g

        path.append(w.copy())


    path = np.array(path)

    plt.plot(path[:, 0], path[:, 1], label=name)
```

```
plt.xlabel("Weight 1")
plt.ylabel("Weight 2")
plt.title("Optimiser Paths")
plt.legend()
plt.grid()
plt.show()

# Keras optimiser comparison
np.random.seed(42)
X = np.random.rand(200, 4)
Y = 5*X[:, 0] + 3*X[:, 1] + 2*X[:, 2] + X[:, 3]

optimizers = [
    ("SGD", tf.keras.optimizers.SGD(0.01)),
    ("Momentum", tf.keras.optimizers.SGD(0.01, momentum=0.9)),
    ("Adam", tf.keras.optimizers.Adam(0.01))
]

for name, opt in optimizers:
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(1)
    ])
    model.compile(optimizer=opt, loss="mse")
    history = model.fit(X, Y, epochs=30, verbose=0)
    plt.plot(history.history["loss"], label=name)
```

```
plt.xlabel("Epochs")

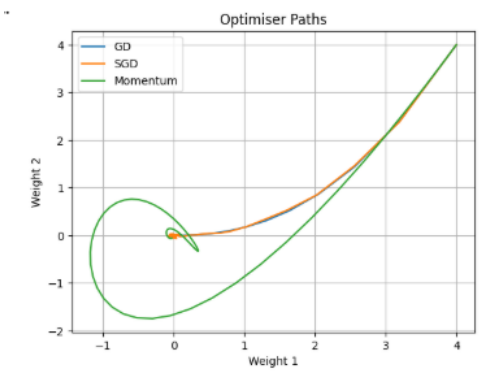
plt.ylabel("Loss")

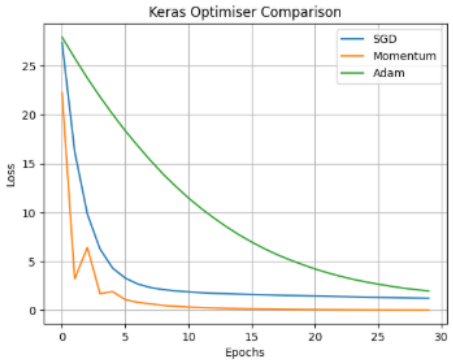
plt.title("Keras Optimiser Comparison")

plt.legend()

plt.grid()

plt.show()
```





Result

Thus, GD, SGD and Momentum were implemented from scratch. SGD, Momentum and Adam were compared in Keras using their training loss curves.

CO2 – NEURAL NETWORKS AND CNNs

EXPERIMENT 7: PERCEPTRON AND FORWARD PROPAGATION

Aim

To implement a single-layer perceptron using NumPy and demonstrate weighted sum, step activation, forward propagation and the perceptron learning rule.

Algorithm

24. Create a small loan approval dataset.

25. Initialise weights and bias.
26. Calculate weighted sum and step activation.
27. Update weights using the perceptron learning rule.
28. Plot the learned decision boundary.
29. Test the perceptron on XOR data and compare step, sigmoid and ReLU activations.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt



# Loan data: 0 = Reject, 1 = Approve

X = np.array([[1,1], [2,1], [1,2], [3,2],

              [2,3], [3,3], [4,3], [4,4]])

y = np.array([0,0,0,0,1,1,1,1])



w = np.zeros(2)

b = 0

lr = 0.1



for epoch in range(30):

    for xi, target in zip(X, y):

        out = int(np.dot(xi, w) + b >= 0)

        error = target - out

        w += lr*error*xi

        b += lr*error



print("Weights:", w, "Bias:", b)

print("Predictions:",

      [int(np.dot(xi,w)+b >= 0) for xi in X])



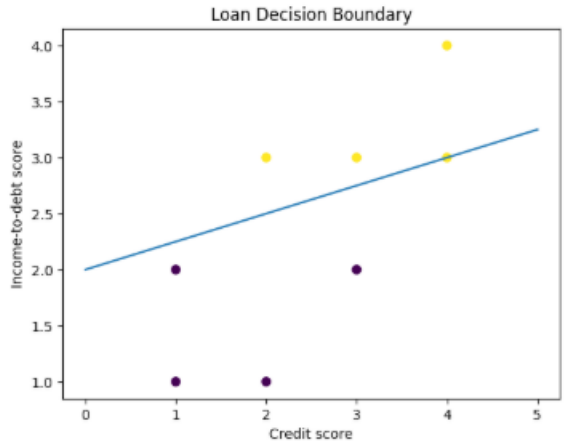
plt.scatter(X[:,0], X[:,1], c=y)
```

```
xx = np.linspace(0, 5, 100)
if w[1] != 0:
    plt.plot(xx, -(w[0]*xx+b)/w[1])
plt.title("Loan Decision Boundary")
plt.xlabel("Credit score")
plt.ylabel("Income-to-debt score")
plt.show()


# XOR test: a single perceptron cannot learn XOR
XOR = np.array([[0,0], [0,1], [1,0], [1,1]])
YOR = np.array([0,1,1,0])
w2 = np.zeros(2)
b2 = 0
for epoch in range(30):
    for xi, target in zip(XOR, YOR):
        out = int(np.dot(xi,w2)+b2 >= 0)
        error = target-out
        w2 += lr*error*xi
        b2 += lr*error
print("XOR predictions:",
      [int(np.dot(xi,w2)+b2 >= 0) for xi in XOR])


z = np.array([-2,-1,0,1,2])
print("Step:", (z >= 0).astype(int))
print("Sigmoid:", 1/(1+np.exp(-z)))
print("ReLU:", np.maximum(0,z))
```

```
Weights: [-0.1  0.4] Bias: -0.7999999999999999
Predictions: [0, 0, 0, 0, 1, 1, 1, 1]
```




```
XOR predictions: [1, 1, 0, 0]
Step: [0 0 1 1 1]
Sigmoid: [0.11920292 0.26894142 0.5        0.73105858 0.88079708]
ReLU: [0 0 0 1 2]
```

Result

Thus, a single-layer perceptron was trained for loan approval. Its limitation in learning the nonlinear XOR pattern was demonstrated.

EXPERIMENT 8: TWO-LAYER NEURAL NETWORK WITH BACKPROPAGATION

Aim

To build a two-layer neural network using NumPy and train it on the XOR dataset using forward propagation and manual backpropagation.

Algorithm

30. Create the XOR input and output data.
31. Initialise weights and biases.
32. Perform forward propagation using sigmoid activation.
33. Calculate the output error.
34. Perform backward propagation using the chain rule.
35. Update weights and biases using gradient descent.
36. Plot the loss against epochs and display predictions.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt



np.random.seed(1)

X = np.array([[0,0], [0,1], [1,0], [1,1]])
```

```
Y = np.array([[0], [1], [1], [0]])


W1 = np.random.randn(2, 4)
W2 = np.random.randn(4, 1)
b1 = np.zeros((1, 4))
b2 = np.zeros((1, 1))


sigmoid = lambda x: 1/(1+np.exp(-x))
losses = []


for epoch in range(10000):
    # Forward propagation
    H = sigmoid(X @ W1 + b1)
    O = sigmoid(H @ W2 + b2)


    losses.append(np.mean((Y-O)**2))

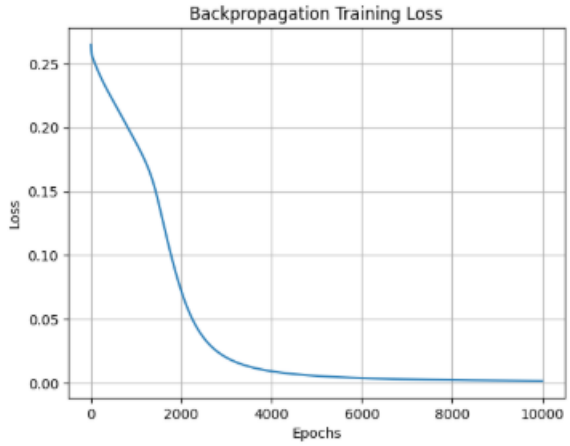

    # Backpropagation
    d2 = (O-Y)*O*(1-O)
    d1 = (d2 @ W2.T)*H*(1-H)


    W2 -= 0.5*(H.T @ d2)/4
    W1 -= 0.5*(X.T @ d1)/4
    b2 -= 0.5*np.mean(d2, axis=0, keepdims=True)
    b1 -= 0.5*np.mean(d1, axis=0, keepdims=True)


print("Predictions:")
print(O.round(2))
```

```
plt.plot(losses)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Backpropagation Training Loss")
plt.grid()
plt.show()
```

```
Predictions:
[[0.02]
 [0.96]
 [0.96]
 [0.05]]
```



Result

Thus, a two-layer neural network was trained using manual backpropagation, and the loss curve showed the reduction in prediction error for the XOR dataset.

EXPERIMENT 9: MULTILAYER PERCEPTRON IMPLEMENTATION

Aim

To build, train and evaluate a Multilayer Perceptron classifier for customer churn prediction using TensorFlow and Keras.

Algorithm

37. Generate a sample customer dataset with five input features and a churn label.
38. Split the data into training and testing sets.
39. Build an MLP with two ReLU hidden layers and a sigmoid output layer.
40. Train the model using Adam and binary cross-entropy.
41. Predict the test labels.
42. Calculate accuracy, precision, recall and F1-score and display the confusion matrix.

Python Code

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.metrics import (accuracy_score, precision_score,

                             recall_score, f1_score,

                             ConfusionMatrixDisplay)


X, y = make_classification(

    n_samples=500, n_features=5, n_informative=4,

    n_redundant=0, random_state=42)


Xtr, Xte, ytr, yte = train_test_split(

    X, y, test_size=0.2, random_state=42)


model = tf.keras.Sequential([

    tf.keras.layers.Input(shape=(5,)),

    tf.keras.layers.Dense(16, activation="relu"),

    tf.keras.layers.Dense(8, activation="relu"),

    tf.keras.layers.Dense(1, activation="sigmoid")

])


model.compile(optimizer="adam",

              loss="binary_crossentropy",

              metrics=["accuracy"])
```

```
history = model.fit(
    Xtr, ytr, epochs=20,
    validation_split=0.2, verbose=0)

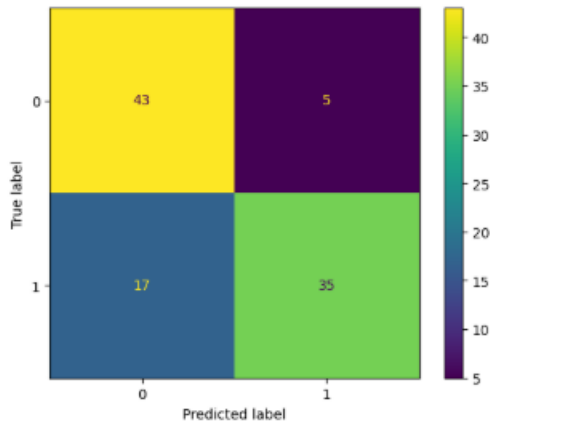

pred = (model.predict(Xte, verbose=0) > 0.5).astype(int)

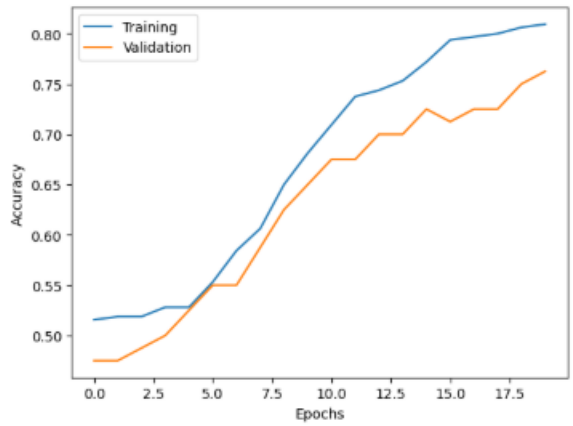

print("Accuracy:", accuracy_score(yte, pred))
print("Precision:", precision_score(yte, pred, zero_division=0))
print("Recall:", recall_score(yte, pred, zero_division=0))
print("F1 Score:", f1_score(yte, pred, zero_division=0))


ConfusionMatrixDisplay.from_predictions(yte, pred)
plt.show()


plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
Accuracy: 0.78
Precision: 0.875
Recall: 0.6730769230769231
F1 Score: 0.7608695652173914
```





Result

Thus, an MLP classifier was trained for sample customer churn data and evaluated using accuracy, precision, recall, F1-score and a confusion matrix.

EXPERIMENT 10: L2, DROPOUT AND EARLY STOPPING COMPARISON

Aim

To compare L2 regularisation, dropout and early stopping techniques for reducing overfitting in a neural network.

Algorithm

43. Generate a sample diabetes-like classification dataset.
44. Standardise the input features and split the dataset.
45. Train a baseline neural network.
46. Train another model using L2 regularisation.
47. Train another model using dropout.
48. Apply early stopping using validation loss.
49. Compare training accuracy, testing accuracy and train-test gap.

Python Code

```
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import regularizers

X, y = make_classification(
    n_samples=500, n_features=8,
    n_informative=5, random_state=42)

X = StandardScaler().fit_transform(X)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=42)

results = []

for method in ["Baseline", "L2", "Dropout", "EarlyStopping"]:
    reg = regularizers.l2(0.001) if method == "L2" else None
    drop = 0.3 if method == "Dropout" else 0

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(8,)),
        tf.keras.layers.Dense(64, activation="relu",
                              kernel_regularizer=reg),
        tf.keras.layers.Dropout(drop),
```

```
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(optimizer="adam",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])

    callbacks = []
    if method == "EarlyStopping":
        callbacks = [tf.keras.callbacks.EarlyStopping(
            patience=3, restore_best_weights=True)]

    model.fit(Xtr, ytr, epochs=30, verbose=0,
              validation_split=0.2, callbacks=callbacks)

    train_acc = model.evaluate(Xtr, ytr, verbose=0)[1]
    test_acc = model.evaluate(Xte, yte, verbose=0)[1]
    results.append([method, train_acc, test_acc,
                    train_acc-test_acc])

df = pd.DataFrame(results, columns=[
    "Method", "Train Accuracy", "Test Accuracy", "Train-Test Gap"])
print(df)
```

	Method	Train Accuracy	Test Accuracy	Train-Test Gap
0	Baseline	0.9425	0.90	0.0425
1	L2	0.9425	0.91	0.0325
2	Dropout	0.9275	0.90	0.0275
3	EarlyStopping	0.9400	0.91	0.0300

Result

Thus, baseline, L2 regularisation, dropout and early stopping models were compared using training accuracy, testing accuracy and train-test gap to study their effect on overfitting.

OVERALL CONCLUSION

Thus, Experiments 3–10 were studied and implemented using Python libraries such as NumPy, SciPy, Pandas, Matplotlib, Scikit-learn, TensorFlow and Keras. These experiments demonstrated probability distributions, Bayes' theorem, entropy, KL divergence, bias–variance analysis, optimisation algorithms, perceptrons, backpropagation, multilayer neural networks and regularisation techniques used in deep learning.